机器人学上机操作实验报告

班级	自动化 3 班
姓名	卞常灿
学号	2023312216
所在学院	智能科学与工程学院
指导教师	杜惠
实验日期	2026 年 5 月 8 日星期五
实验成绩	

编写：杜惠

哈尔滨工业大学（深圳）

1. 机器人关节空间与笛卡尔空间梯形速度规划推导过程

1.1 梯形速度规划数学推导

梯形速度规划的核心是将位移过程分为**匀加速、匀速、匀减速**三个阶段。设总位移为 L，最大速度为 V_{max}，加速度为 a，减速度为 d。

1. **时间参数计算**：
   - 首先计算达到 V_{max} 所需的临界位移 $L_c = \frac{V_{max}^2}{2a} + \frac{V_{max}^2}{2d}$。
   - **情况 A（梯形）**：若 $L > L_c$，则可以达到最大速度。
     - 加速时间 $t_a = \frac{V_{max}}{a}$，减速时间 $t_d = \frac{V_{max}}{d}$。
     - 匀速时间 $t_v = \frac{L - \frac{1}{2}at_a^2 - \frac{1}{2}dt_d^2}{V_{max}}$。
   - **情况 B（三角形）**：若 $L \le L_c$，则无法达到最大速度，需计算实际峰值速度 $V_p = \sqrt{\frac{2Lad}{a+d}}$。
     - 实际时间：$t_a = \frac{V_p}{a}$，$t_d = \frac{V_p}{d}$，$t_v = 0$。
2. **实时状态方程 $q(t)$**：
   - $t \in [0, t_a]$：$q(t) = q_0 + \frac{1}{2}at^2$。
   - $t \in (t_a, t_a + t_v]$：$q(t) = q(t_a) + V_{max}(t - t_a)$。
   - $t \in (t_a + t_v, T]$：$q(t) = q(t_a + t_v) + V_{max}\Delta t - \frac{1}{2}d\Delta t^2$（$\Delta t = t - t_a - t_v$）。

1.2 空间应用差异

本实验中，同一套规划算法在两个空间的物理含义及后续处理完全不同：

- **关节空间（Joint Space）**：
  - **物理含义**：位移 L 代表某一关节角度的改变量 $\Delta\theta_i$。
  - **规划逻辑**：六个关节分别作为独立的标量进行时间规划。
  - **实现特点**：直接生成驱动电机的关节序列，无需逆解。
- **笛卡尔空间（Cartesian Space）**：
  - **物理含义**：位移 L 代表末端法兰中心在世界坐标系下的分量差 $\Delta x, \Delta y, \Delta z$。
  - **推导过程**：
    1. 在笛卡尔坐标系下，通过上述公式生成末端位置序列 $\mathbf{P}(t) = [x(t), y(t), z(t)]^T$。
    2. **逆运动学映射**：由于电机位于关节轴上，必须通过逆解函数 `Ikine6s` 将每一个时刻 t 对应的空间位置 $\mathbf{P}(t)$ 及给定的目标姿态矩阵 R_{target} 转换为对应的六轴角度 $\theta_1(t) \dots \theta_6(t)$。
    3. **姿态约束**：在平移过程中，为了码垛安全，本实验通过旋转矩阵变换，强制末端工具 Z 轴垂直指向下方。

1.3 核心规划代码实现

以下为实现上述数学推导的 `MotionPlanning` 与 `TrajGena` 函数：

```
% Author:
%
% ABSTRACT:  三阶轨迹规划最基本文件 (梯形速度规划 + 时间对齐)
%
function MotionData = MotionPlanning(TotalLength, Vmax, Accel, Decel, Ts)
    N = length(TotalLength);
    MotionData.ta = zeros(1, N); MotionData.tv = zeros(1, N); MotionData.td =
zeros(1, N);
    MotionData.vmax_real = zeros(1, N); MotionData.acc_real = zeros(1, N);
MotionData.dec_real = zeros(1, N);
    MotionData.dir = sign(TotalLength);
    MotionData.Ts = Ts;

    T = zeros(1, N);
    ta = zeros(1, N); tv = zeros(1, N); td = zeros(1, N);

    % 1. 计算各轴独立运行所需的最短时间
    for i = 1:N
        L = abs(TotalLength(i));
        if L < 1e-6
            continue; % 位移极小，不运动
        end
        % 判断距离是否足够达到最大速度
        if L > (Vmax(i)^2)/(2*Accel(i)) + (Vmax(i)^2)/(2*Decel(i))
            ta(i) = Vmax(i)/Accel(i);
            td(i) = Vmax(i)/Decel(i);
            tv(i) = (L - 0.5*Accel(i)*ta(i)^2 - 0.5*Decel(i)*td(i)^2) / Vmax(i);
        else
            % 达不到最大速度，呈三角形规划
            V_peak = sqrt(2*L*Accel(i)*Decel(i)/(Accel(i)+Decel(i)));
            ta(i) = V_peak/Accel(i);
            td(i) = V_peak/Decel(i);
            tv(i) = 0;
        end
        T(i) = ta(i) + tv(i) + td(i);
    end

    % 2. 时间对齐 (Time Alignment): 找出最大时间
    T_max = max(T);
    if T_max == 0, T_max = Ts; end
    MotionData.maxtime = T_max;

    % 3. 按最大时间缩放其他轴的运动参数，实现同步到达
```

```
    for i = 1:N
        if T(i) > 0
            k = T_max / T(i); % 时间拉伸系数
            MotionData.ta(i) = ta(i) * k;
            MotionData.tv(i) = tv(i) * k;
            MotionData.td(i) = td(i) * k;
            % 时间变长，距离不变，需要重新计算对齐后的真实速度与加速度
            MotionData.vmax_real(i) = abs(TotalLength(i)) / (MotionData.ta(i)/2 +
MotionData.tv(i) + MotionData.td(i)/2);
            MotionData.acc_real(i) = MotionData.vmax_real(i) / MotionData.ta(i);
            MotionData.dec_real(i) = MotionData.vmax_real(i) / MotionData.td(i);
        end
    end
end
```

```
% Author:
%
% ABSTRACT:  根据时间和路径公式参数结构体计算当前时间点，关节位置，速度和加速度
%
function [q, qd, qdd] = TrajGena(time, MotionData, loc1, loc2)
    N = length(loc1);
    q = zeros(1,N); qd = zeros(1,N); qdd = zeros(1,N);

    for i = 1:N
        dir = MotionData.dir(i);
        ta = MotionData.ta(i); tv = MotionData.tv(i); td = MotionData.td(i);
        v = MotionData.vmax_real(i); a = MotionData.acc_real(i); d =
MotionData.dec_real(i);

        if time <= ta
            % 加速段
            q(i) = loc1(i) + dir * 0.5 * a * time^2;
            qd(i) = dir * a * time;
            qdd(i) = dir * a;
        elseif time <= ta + tv
            % 匀速段
            q(i) = loc1(i) + dir * (0.5 * a * ta^2 + v * (time - ta));
            qd(i) = dir * v;
            qdd(i) = 0;
        elseif time <= ta + tv + td
            % 减速段
            t_dec = time - ta - tv;
            q(i) = loc1(i) + dir * (0.5 * a * ta^2 + v * tv + v * t_dec - 0.5 * d
* t_dec^2);
            qd(i) = dir * (v - d * t_dec);
            qdd(i) = -dir * d;
        else
```

```
            % 停止阶段
            q(i) = loc2(i);
            qd(i) = 0;
            qdd(i) = 0;
        end
    end
end
```

---

2. 轨迹规划是否考虑到了对齐时间及对齐方法

是，轨迹规划严格考虑了对齐时间（Time Alignment）。

2.1 对齐方法推导

当机器人的 X, Y, Z 三轴需要同时移动到目标点时，由于各分量的位移长度不一，按各自限速运动会导致有的轴先到、有的轴后到，使合成轨迹变成折线。本实验采用了**全局最大时间拉伸法**：

1. **求解基准时间**：计算所有分量轴独立完成规划所需的时间 T_i，取其最大值作为全局同步周期：$T_{max} = \max(T_i)$。
2. **重构时间常数**：引入时间拉伸系数 $k_i = T_{max} / T_i$。
3. **参数同步缩放**：保持各段位移比例不变，将该轴对应的 t_a, t_v, t_d 同比例放大为 $t_{a_new} = t_a \cdot k_i$ 等。
4. **修正速度与加速度**：根据拉伸后的时间，反算真实的 V_{real} 与 Acc_{real}，从而保证所有坐标分量同时起步、同时停止。

2.2 代码逻辑体现

```
% MotionPlanning.m 中的对齐片段
% 2. 时间对齐 (Time Alignment): 找出最大时间
T_max = max(T);
if T_max == 0, T_max = Ts; end
MotionData.maxtime = T_max;

% 3. 按最大时间缩放其他轴的运动参数，实现同步到达
for i = 1:N
    if T(i) > 0
        k = T_max / T(i); % 时间拉伸系数
        MotionData.ta(i) = ta(i) * k;
        MotionData.tv(i) = tv(i) * k;
        MotionData.td(i) = td(i) * k;
        % 时间变长，距离不变，需要重新计算对齐后的真实速度与加速度
        MotionData.vmax_real(i) = abs(TotalLength(i)) / (MotionData.ta(i)/2 +
MotionData.tv(i) + MotionData.td(i)/2);
        MotionData.acc_real(i) = MotionData.vmax_real(i) / MotionData.ta(i);
        MotionData.dec_real(i) = MotionData.vmax_real(i) / MotionData.td(i);
    end
```

```
end
```

---

3. 码垛功能实现过程描述

3.1 文字描述

码垛任务通过“门字形”轨迹实现物料从抓取点 A 到放置点 B 的搬运。具体流程如下：

1. **路径定义**：在 `Exp4_Main.m` 中定义了 6 个关键笛卡尔点位。序列为：`P1_up`(抓取上方) → `P2_pck`(抓取点) → `P1_up` → `P3_up`(放置上方) → `P4_plc`(放置点) → `P3_up`。
2. **轨迹生成**：调用 `MotionPlanning` 和 `TrajGena` 对上述每一段直线进行 1ms 步长的梯形速度规划插补，生成末端位置矩阵。
3. **坐标映射与优选**：针对生成的每一组笛卡尔点位，设定目标旋转矩阵 R_{target}（确保末端垂直向下），调用 `Ikine6s.m` 求解关节角度。
    - **姿态优选**：逆解过程中通过 `th3 = atan2(sqrt(1 - D^2), D)` 强制选取“肘部朝上”解，避免与实验台碰撞。
4. **数据后处理**：
    - 使用 15 点滑动平均滤波（`smoothdata`）消除分段边界因数值求导产生的加速度尖峰。
    - 将关节弧度转换为角度（`rad2deg`），按 100ms 降采样率导出为 `joint_pos.txt`。
5. **实机执行**：采用 **Ping-pong Buffer (PPB)** 机制。首先通过 `PPB.JStartPos` 指令使机器人从手动示教位置平滑移动到轨迹起点，然后执行 `PPB.Run` 高速完成码垛动作 。

3.2 码垛主程序与逆解集成实现

```
% ==========================================
% Exp4_Main.m: 笛卡尔空间梯形速度规划与码垛任务
% ==========================================
clear; clc;
Initialization; % 调用实验三的初始化参数，获取 config, g0, Xi

% 1. 定义码垛的笛卡尔关键点位 (X, Y, Z)
% 提取自实验二的门形轨迹：抓取点P_pick 与 放置点P_place
P_home = [400, 0, 800]; % 初始安全高点
P1_up  = [541.8, -413.8, 430.0]; % 抓取点正上方
P2_pck = [541.8, -413.8, 401.5]; % 抓取点
P3_up  = [446.8, -413.8, 430.0]; % 放置点正上方
P4_plc = [446.8, -413.8, 396.5]; % 放置点

% 门形轨迹路点顺序
% Waypoints = [P_home; P1_up; P2_pck; P1_up; P3_up; P4_plc; P3_up; P_home];
Waypoints = [P1_up; P2_pck; P1_up; P3_up; P4_plc; P3_up];
num_segments = size(Waypoints, 1) - 1;
```

```
% 2. 笛卡尔空间限制参数
Vmax  = [50, 50, 50];     % 最大速度降到 50 mm/s
Accel = [100, 100, 100]; % 加减速度降到 100
Decel = [100, 100, 100];
Ts = 0.001; % 1ms 仿真步长

% --- 3. 生成笛卡尔直线插补轨迹 (修复数组拼接导致的加速度尖峰) ---
Cart_Traj = Waypoints(1, :); % 显式放入起点，作为整个轨迹的绝对第一帧

for seg = 1:num_segments
    loc1 = Waypoints(seg, :);
    loc2 = Waypoints(seg+1, :);
    TotalLength = loc2 - loc1;

    % 调用梯形速度规划 (内置时间对齐)
    MotionData = MotionPlanning(TotalLength, Vmax, Accel, Decel, Ts);

    % 严格计算当前段所需的总插补步数 (四舍五入，消除浮点截断误差)
    num_steps = round(MotionData.maxtime / Ts);

    % 核心修复：从 k=1 (即 t=Ts) 开始循环，坚决跳过 t=0!
    % 因为上一段的最后一个点，刚好就是这一段的 t=0 点，这样就完美避免了重复和阶跃。
    for k = 1:num_steps
        time = k * Ts;
        [pos, ~, ~] = TrajGena(time, MotionData, loc1, loc2);
        Cart_Traj = [Cart_Traj; pos];
    end
end

% ============================================================
% 【4. 逆运动学求解...】(修正后：选取工具向下的优选姿态)
% ============================================================
num_pts = size(Cart_Traj, 1);
t = (0 : num_pts-1)' * Ts;
q = zeros(num_pts, 6);

% --- 修正：为整个码垛过程设定工具垂直向下的目标姿态 ---
% 指导书参数显示，初始位姿工具Z轴朝上。
% 我们将 $g_0$ 绕自身 Y 轴旋转 180 度 (pi rad) 来使工具指向下方。
R_g0 = g0(1:3, 1:3);
R_down = [cos(pi) 0 sin(pi); 0 1 0; -sin(pi) 0 cos(pi)];
R_target = R_g0 * R_down; % 正确的工具垂直向下姿态

disp('正在进行逆运动学解算，请稍候...');
for i = 1:num_pts
    % 组装当前时刻的目标齐次变换矩阵 T
    T_target = [R_target, Cart_Traj(i,:)'; 0 0 0 1];
```

```
    % 调用你的优选单组逆解函数 Ikine6s
    % 算法应已优先选取"肘部朝上"解。
    q(i, :) = Ikine6s(T_target, config);
end

% 差分计算速度和加速度矩阵 (喂给 Simulink 模型)
qd = zeros(num_pts, 6); qdd = zeros(num_pts, 6);
qd(2:end, :) = diff(q) / Ts;
qdd(2:end, :) = diff(qd) / Ts;

% 【终极修复】：加入 15 点滑动平均滤波
% 这模拟了真实控制器内部的低通滤波，专门消除离散求导在分段边界处产生的脉冲尖峰
qd = smoothdata(qd, 'movmean', 15);
qdd = smoothdata(qdd, 'movmean', 15);

% 5. 导出供真机 PPB 模块使用的点位文件 (data.txt)
% 指导书规定：位置 x, y, z，欧拉角 rz, ry, rz
fileID = fopen('cart_pos.txt', 'w');
for i = 1:1:num_pts % 每隔1ms下一个点
    % 姿态保持 180, 0, 0 对应基准垂直姿态
    fprintf(fileID, '%.3f %.3f %.3f 180.0 0.0 0.0\n', Cart_Traj(i,1),
Cart_Traj(i,2), Cart_Traj(i,3));
end
fclose(fileID);

disp('规划完成！数据已加载到工作区。');
disp('>> 请打开 HL0905_1.slx 或对应仿真模型点击 [运行/Run] 查看动画。');

% --- 附加：导出供真机 PPB 使用的 关节空间 点位文件 (joint.txt) ---
% 必须将弧度(q)转换为角度(degree)！
fileID_j = fopen('joint_pos.txt', 'w');
for i = 1:1:num_pts
    % 使用 rad2deg 将弧度转为角度输出
    q_deg = rad2deg(q(i, :));
    fprintf(fileID_j, '%.4f %.4f %.4f %.4f %.4f %.4f\n', ...
            q_deg(1), q_deg(2), q_deg(3), q_deg(4), q_deg(5), q_deg(6));
end
fclose(fileID_j);
disp('附加的关节空间点位 joint_pos.txt 也已生成（全为角度制）。');
```

3.3 码垛逻辑流程图


六轴工业机器人码垛任务离线规划与上机控制流程图
第一阶段：MATLAB 离线算法规划
A. 开始实验
B. 环境参数初始化
导入机器人连杆长度、建立旋量矩阵与初始位姿g0
C. 定义门字形笛卡尔路径
设定起点 -> 物料A抓取点 -> 物料B放置点的6段关键空间点位
D. 带时间对齐的梯形速度规划
计算各轴最短运动时间，提取全局最大时间拉伸以实现轴同步
E. 笛卡尔空间路径插补
按 1ms 步长生成平滑的 X-Y-Z 空间位置序列
X
Y
F. 姿态约束与逆向运动学解算
强制工具垂直向下，调用 Ikine6s 优选"肘部朝上"安全构型，
映射出关节弧度序列
IK
G. 滤波平滑与降采样处理
使用15点滑动平均消除加速度跃变毛刺，
将弧度转换为角度并按 100ms 降采样
H. 导出关节点位文件 joint_pos.txt
TXT
数据跨端传递
第二阶段：实体机器人 PPB 上机执行
I. 文件网络上传
通过 FTP 将文件传入机器人控制器 /data 目录
FTP
J. 手动防撞示教
在手动模式下，将机械臂移至无干涉、开阔的安全高位
K. 开启 PPB 模式并读取文件
切换至自动模式，终端执行 PPB.Enable 与 PPB.ReadFile 指令
L. 自动接驳轨迹起点
执行 PPB.JStartPos，机器人从当前安全位置自动平滑回溯至轨迹的绝对起点
M. 执行连续码垛运动
执行 PPB.Run，底层伺服按样条插补完成高速的门字形抓放作业
N. 任务圆满结束